



Software Factory en la Era de la IA

Gestion Proyectos

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Planificacion Agil

Planificación Ágil

La planificación ágil es un enfoque iterativo e incremental para planificar trabajo de software, adaptándose al cambio en lugar de seguir un plan rígido.

Niveles de planificación

Nivel	Horizonte	Participantes	Artefacto principal
Product Vision	2-5 años	Product, Leadership	Product roadmap
Roadmap	6-18 meses	Product, Engineering	Quarterly roadmap
Release Planning	1-3 meses	Squad completo	Release plan
Sprint Planning	1-2 semanas	Equipo Scrum	Sprint backlog
Daily	1 día	Equipo Scrum	Sprint board

Product Roadmapping

Un roadmap comunica la dirección estratégica del producto. No es una lista de features, sino una narrativa de valor.

Tipos de roadmap

Tipo	Uso principal	Audiencia
Now-Next-Later	Comunicación ágil, bajo detalle	Equipos, stakeholders
Timeline	Compromisos fechados	Leadership, clientes
Outcome-based	Orientado a métricas	Product, executives
Theme-based	Alineación estratégica	Organización completa

Plantilla Now-Next-Later

```
NOW (este sprint/quarter)
├─ Feature A – impacto: alto, confianza: alta
├─ Feature B – impacto: alto, confianza: media
└─ Tech debt: migración DB – urgencia: crítica

NEXT (próximos 1-2 quarters)
├─ Feature C – exploración en curso
└─ Feature D – depende de A

LATER (horizonte 3-6+ meses)
```

- └ Feature E – viable, no priorizado
- └ Feature F – idea early stage

PI Planning (Program Increment Planning)

Evento central de SAFe donde múltiples equipos sincronizan su plan para un Program Increment (8-12 semanas).

Estructura del PI Planning

Fase	Duración	Actividad
Prep	1 día	Visión, arquitectura, roadmap
Day 1 AM	4h	Business context, product vision
Day 1 PM	4h	Team breakouts: estimar, identificar dependencias
Day 2 AM	4h	Adjustments, plan final, risk/dependency discussion
Day 2 PM	2h	Confidence vote, management review

Confidence Vote (Fist of Five)

Voto	Significado
👊 (5)	Totalmente confiado, puedo comprometerme
4	Confianza alta, menores preocupaciones
3	Neutral, necesito hablar
2	Preocupaciones serias
1	No puedo comprometerme con este plan

Si el promedio es < 3, se re-plana.

Release Planning

Planificar releases concretas con objetivos claros de negocio.

Checklist de Release Planning

- ☐ Objetivo de negocio definido y comunicado
- ☐ Features priorizadas con estimaciones de alto nivel
- ☐ Dependencias identificadas y gestionadas
- ☐ Riesgos principales documentados
- ☐ Criterios de Aceptación claros
- ☐ Stakeholders informados y alineados
- ☐ Capacidad del equipo verificada

Horizon Planning

Divide el roadmap en horizontes para balancear certeza y flexibilidad.

Horizonte	Detail level	Re-planning	Scope
H1 (0-3 meses)	Story level	Cada sprint	Detallado
H2 (3-6 meses)	Feature level	Mensual	Moderado
H3 (6-18 meses)	Theme level	Trimestral	Estratégico

Anti-patterns

- **Commitment-driven planning:** comprometer fechas sin estimación técnica
- **Waterfall disfrazado:** planificar todo el quarter de una vez sin revisar
- **Roadmap como lista de features:** en lugar de outcomes y métricas
- **Planning sin capacity:** asumir 100% disponibilidad del equipo
- **Ignoring tech debt:** no incluir mantenimiento en la planificación

Métricas de planificación

Métrica	Fórmula	Target típico		
Plan accuracy	(completed / committed) × 100	> 80%		
Forecast accuracy	1 - (	estimado - real	/ real)	> 70%
Sprint goal rate	sprints con goal / total sprints	> 85%		
On-time release	releases on-time / total releases	> 90%		



Estimacion Tecnicas

Técnicas de Estimación

Estimar es predecir el esfuerzo, tiempo o costo de un trabajo. Ninguna técnica elimina la incertidumbre, pero todas la hacen más manejable.

Principios fundamentales

- Estimar es **proyectar**, no comprometer
- Mejor estimar en **rangos** que en valores puntuales
- Las estimaciones se **calibran** con datos históricos
- Incluir siempre el **factor de incertidumbre** ($\pm$)
- Re-estimar cuando cambia el **scope** o la información

Planning Poker

Técnica de estimación colaborativa usando cartas con valores de la serie Fibonacci modificada.

Flujo

1. Product Owner presenta la historia de usuario
2. Cada miembro selecciona una carta (oculta)
3. Todos revelan simultáneamente
4. Si hay divergencia > 2 puntos, se discute
5. Se repite hasta lograr consenso

Valores típicos

Carta	Significado
0	Trivial, ya hecho
1	Muy pequeño (< 1 día)
2	Pequeño (1-2 días)
3	Medio-pequeño (2-3 días)
5	Medio (3-5 días)
8	Medio-grande (1 semana)
13	Grande (1-2 semanas)
20	Muy grande, necesita splitting
40	Enorme, decomponer obligatorio

Carta	Significado
100	Masivo, no estimable aún
?	No tengo información
☕	Necesito una pausa / discusión

Tips para Planning Poker

- Estimadores con **experiencia técnica** deben participar
- El **PO no vota** pero puede aclarar requisitos
- Si todos votan igual en la primera ronda → aceptar
- Usar datos de **velocity** para calibrar la escala

T-Shirt Sizing

Asigna tamaños relativos: XS, S, M, L, XL, XXL. Útil para estimación de alto nivel.

Mapeo típico

Talla	Story Points	Days approx	Uso
XS	1	0.5-1	Quick fix, config
S	2-3	1-3	Feature pequeña
M	5	3-5	Feature estándar
L	8-13	5-10	Feature compleja
XL	20-40	10-20	Epic que necesita splitting
XXL	40+	20+	Initiative, no estimar aún

Cuando usar T-Shirt Sizing

Situación	Recomendación
Roadmap de alto nivel	T-Shirt
PI Planning	T-Shirt
Sprint Planning	✗ Story Points
Comparación entre equipos	T-Shirt
Presupuesto preliminar	T-Shirt
Sprint commitment	✗ Story Points

Reference Class Forecasting (RCF)

Basado en datos históricos reales, no en estimaciones subjetivas.

Pasos

- 1. **Identificar clase de referencia:** encontrar proyectos similares en el historial
- 2. **Recopilar datos:** duración real, costo, scope de la clase
- 3. **Estimar distribución:** calcular percentiles (P10, P50, P90)
- 4. **Seleccionar estimación:** usar P50 como punto medio, P85 como compromiso

Ejemplo

Proyecto similar	Duración real
API migration v1	12 semanas
API migration v2	15 semanas
API migration v3	10 semanas
Promedio	12.3 semanas
Desvío estándar	2.5 semanas

Estimación nueva: 12 semanas (P50), 15 semanas (P85)

Estimación por analogía

Comparar directamente con historial de stories similares ya completadas.

Template

```
Historia nueva: "Implementar pagos con Stripe"  
└─ Similar a: "Implementar pagos con PayPal" → 8 SP  
└─ Diferencia: Stripe tiene más documentación → -1 SP  
└─ Factor riesgo: primera integración con 3DS → +2 SP  
└─ Estimación final: 9 SP (redondear a 8 o 13)
```

Estimación de three-point (PERT)

Usa tres estimaciones para calcular un valor esperado con incertidumbre.

```
Optimista (O) = mejor caso  
Most Likely (M) = caso más probable  
Pessimistic (P) = peor caso  
  
Expected = (O + 4M + P) / 6  
Std Dev = (P - O) / 6
```

Story	O	M	P	Expected	Std Dev
Login	2	3	7	3.5	0.83
Pago	5	8	20	9.2	2.5
Report	3	5	12	5.8	1.5

Errores comunes

Error	Descripción	Cómo evitarlo
Anchoring	Primera estimación domina el grupo	Votar simultáneamente
Commitment bias	Presión por comprometerse	Estimar $\neq$ comprometer
Hofstadter's Law	Siempre toma más de lo esperado	Agregar buffer ($\times 1.3$)
Precision illusion	Dar números exactos innecesarios	Usar rangos
Availability bias	Recordar el último incidente grande	Mirar historial completo

Métricas de calibración

Métrica	Fórmula	Target	
Estimation accuracy	estimado - real	/ real $\times$ 100	< 30%
Bias ratio	overestimates / underestimates	0.8-1.2	
Confidence interval	% de estimaciones dentro del rango	> 80%	
Calibration score	predicciones correctas / total	> 70%	

Story Points Tshirt

Story Points vs T-Shirt Sizing

Comparación de las dos técnicas de estimación relativa más usadas en equipos de software ágiles.

Definiciones

Aspecto	Story Points	T-Shirt Sizing
Escala	Numérica (Fibonacci: 1,2,3,5,8,13...)	Cualitativa (XS, S, M, L, XL, XXL)
Basis	Esfuerzo + complejidad + incertidumbre	Tamaño relativo percibido
Precisión	Alta (numérica)	Baja (categórica)
Velocidad	Requiere calibración del equipo	Inmediata de usar
Comparabilidad	Entre sprints del mismo equipo	Entre equipos sin calibración

Matriz de comparación

Criterio	Story Points	T-Shirt Sizing	Ganador
Sprint planning	Ideal	⚠️ Poco preciso	SP
Roadmap alto nivel	⚠️ Muy detallado	Ideal	T-Shirt
Velocity tracking	Requerido	❌ No aplica	SP
Cross-team estimation	⚠️ Requiere calibración	Funciona directo	T-Shirt
Nuevo equipo	⚠️ Calibración necesaria	Listo para usar	T-Shirt
Presupuesto/timeline	⚠️ Conversión necesaria	Mapeo directo	T-Shirt
Retrospectivas	Datos concretos	⚠️ Díficil medir	SP
Comunicación con stakeholders	❌ Técnico	Intuitivo	T-Shirt

Story Points — Detalle

Escala Fibonacci modificada

Puntos	Rango de días	Complejidad
1	< 1 día	Trivial
2	1-2 días	Simple

Puntos	Rango de días	Complejidad
3	2-3 días	Estándar
5	3-5 días	Moderada
8	5-8 días	Compleja
13	8-13 días	Muy compleja
21+	> 13 días	Requiere splitting

Ventajas

- Base para **velocity** y capacity planning
- Permite **forecasting** con datos históricos
- Captura tres dimensiones: esfuerzo, complejidad, incertidumbre
- Facilita **sprint commitment** basado en capacidad

Desventajas

- Requiere **período de calibración** (3-5 sprints)
- Los números son **abstractos** para stakeholders no técnicos
- Pueden convertirse en **métrica de performance** (mal uso)
- Comparación entre equipos es **poco fiable** sin normalización

T-Shirt Sizing — Detalle

Escala estándar

Talla	Story Points equiv.	Días approx	Criterio
XS	1	0.5-1	Config, fix trivial
S	2-3	1-3	Feature pequeña
M	5	3-5	Feature estándar
L	8-13	5-10	Feature grande
XL	20-40	10-20	Epic / splitting
XXL	40+	20+	Initiative

Ventajas

- **Inmediato**: sin período de calibración
- **Intuitivo**: stakeholders lo entienden al instante
- **Cross-team**: funciona sin normalización
- **Adecuado** para 早期阶段 de un equipo o proyecto

Desventajas

- No genera **velocity** numérica
- No permite **forecasting** detallado
- Los rangos son **muy amplios** (M puede ser 3 o 5 días)
- Difícil de **retroalimentar** en retrospectivas

Cuándo usar cada uno

Escenario	Recomendación	Razón
Sprint planning semanal	Story Points	Necesitas compromiso numérico
PI Planning (SAFE)	T-Shirt → convertir	Alto nivel, muchos equipos
Roadmap de 12 meses	T-Shirt	No se necesita precisión
Nuevo equipo (< 3 sprints)	T-Shirt	Sin datos históricos
Equipo estable (> 5 sprints)	Story Points	Velocity disponible
Presupuesto preliminar	T-Shirt + benchmark	Estimación rough
Negociación con cliente	T-Shirt	Más fácil de comunicar
Forecasting de release	Story Points	Requiere datos numéricos

Alternativas a ambos

Técnica	Descripción	Cuándo usar
Throughput	Contar historias completadas por semana	Equipos maduros, stories similares
Cycle Time	Medir tiempo real de entrega	Kanban, continuous delivery
Ideal Days	Días ideales de trabajo (sin interrupciones)	Equipos con poca variabilidad
No estimate	No estimar, medir throughput directo	Teams maduros, flow-based
Dot Voting	Votos para priorizar, no estimar	Priorización, no tamaño

Estrategia híbrida recomendada

1. backlog → T-Shirt sizing (rápido, early stage)
2. sprint ready → Story Points (precisión)
3. release plan → Story Points → velocity-based forecast
4. roadmap → T-Shirt (comunicar con stakeholders)

Errores comunes

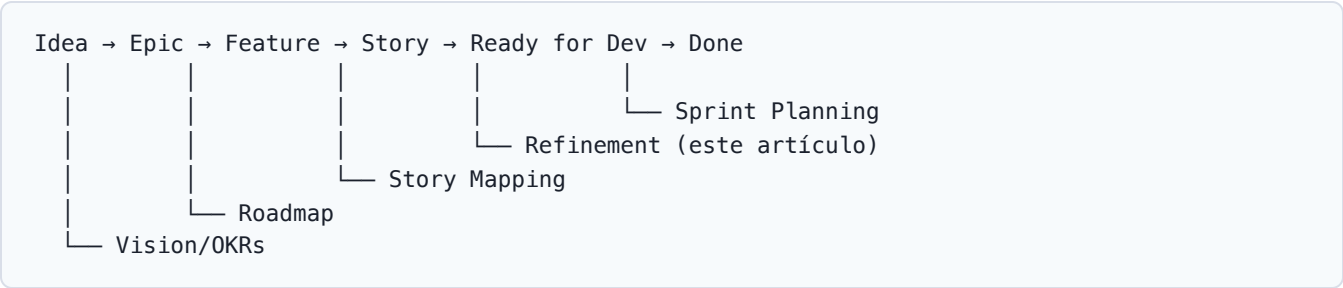
- Usar story points como **métrica de productividad** individual
- Convertir points → hours (pierde el valor relativo)
- No recalibrar la escala cuando cambia el equipo
- Usar T-Shirt sizing en sprint planning (poco preciso)
- Esperar consistencia perfecta entre talla y puntos

Backlog Grooming

Backlog Grooming / Refinement

El refinement es el proceso continuo de mantear el product backlog listo para desarrollo, con historias claras, estimadas y priorizadas.

Ciclo de vida de una historia



INVEST Criteria

Cada historia debe cumplir los criterios INVEST para estar "ready":

Criterio	Significado	Pregunta clave
Independent	No depende de otras stories	¿Se puede entregar sola?
Negotiable	El scope es negociable	¿Hay más de una forma de hacerlo?
Valuable	Aporta valor al usuario/negocio	¿Quién se beneficia?
E stimable	Se puede estimar razonablemente	¿Tenemos suficiente info?
Small	Lo suficientemente pequeña para un sprint	¿Cabe en 3-5 días?
Testable	Tiene criterios de aceptación claros	¿Cómo sabemos que funciona?

Checklist de Definition of Ready

- ☐ Historia de usuario escrita: "Como [rol], quiero [acción] para [beneficio]"
- ☐ Criterios de aceptación definidos (mínimo 2-3)
- ☐ Dependencias identificadas y desbloqueadas
- ☐ Diseño/UX mocks disponibles si aplica
- ☐ Estimación completada (story points o T-Shirt)
- ☐ Datos de ejemplo o seed data definidos
- ☐ Technical approach revisado por el equipo

Story Mapping

Técnica para organizar historias por user journey y prioridad.

Estructura

User Journey (horizontal)	→	Login	→	Search	→	Purchase	→	Profile
Release 3 (MVP)				MVP		MVP		
Release 2 (Foundation)		Basic		Basic		Basic		Basic
Release 1 (Core)		Full		Full		Full		Full
Walking Skeleton → Minimal → Full Feature								

Pasos para Story Mapping

1. **Escribir user activities** (ejes horizontales)
2. **Descomponer en user tasks** (bajo cada activity)
3. **Colocar en orden de user journey** (izquierda → derecha)
4. **Dibujar líneas de release** (horizontal: arriba = más prioritario)
5. **Identificar walking skeleton**: el mínimo para que el sistema funcione end-to-end
6. **Llenar releases**: de abajo hacia arriba, priorizando valor

Story Slicing (INVEST aplicado)

Patrones de slicing

Patrón	Ejemplo	Resultado
By workflow step	"Proceso de pago completo" → "Validar tarjeta", "Cobrar", "Confirmar"	Stories por paso
By data type	"Gestionar usuarios" → "Crear", "Editar", "Eliminar"	CRUD separado
By platform	"App responsive" → "Desktop", "Mobile", "Tablet"	Por dispositivo
By business rule	"Cálculo de impuestos" → "IVA básico", "Exenciones", "Multi-país"	Por regla
By interface	"Sistema de notificaciones" → "Email", "Push", "SMS"	Por canal

INVEST aplicado al slicing

Epic: "Sistema de pagos"
├ Story XL: "Acepta todas las formas de pago" ❌ (no INVEST)
├ Story M: "Acepta tarjeta de crédito" ✔️
├ Story M: "Acepta PayPal" ✔️
├ Story S: "Acepta transferencia bancaria" ✔️
└ Story M: "Acepta criptomonedas" ✔️

Técnicas de Refinement

Refinement meeting (recomendado)

Aspecto	Recomendación
Frecuencia	1-2 veces por sprint, 30-60 min
Participantes	Dev team + PO + Tech Lead
Input	Top 10-15 items del backlog
Output	Stories listas para sprint siguiente
Timebox	Máximo 10% de capacity del sprint

Refinement asíncrono

- PO escribe stories con acceptance criteria
- Dev team comenta en la herramienta (Jira, Linear)
- Se resuelven dudas antes del refinement meeting
- Ideal para equipos distribuidos

Anti-patterns

Anti-pattern	Problema	Solución
Sprint Planning como refinement	Planning se alarga 4+ horas	Refinar 1-2 sprints adelante
Stories gigantes	No caben en un sprint	Aplicar story slicing
Acceptance criteria vacíos	Ambigüedad, rework	PO escribe AC antes de refinement
Refinement solo del PO	Falta perspectiva técnica	Incluir devs en refinement
Backlog infinito	Items obsoletos acumulados	Limpiar backlog trimestralmente

Métricas de calidad del backlog

Métrica	Target	Cómo medir
% Ready stories	> 70% del top 20	Stories que cumplen INVEST
Refinement velocity	10-15 items/sprint	Items refinados por sesión
Sprint Planning time	< 2 horas	Para sprints de 2 semanas
Re-estimation rate	< 15%	Stories que cambian de estimación
Backlog age	< 90 días top items	Días desde creación

Gestión de Riesgos en Software

Framework de gestión de riesgos



Evaluación de probabilidad e impacto

Nivel	Valor	Descripción
Muy baja	1/5	< 10% chance de ocurrir
Baja	2/5	10-30%
Media	3/5	30-60%
Alta	4/5	60-80%
Muy alta	5/5	> 80%

Impacto

Nivel	Valor	Schedule	Cost	Quality
Negligible	1/5	< 1 semana	< 5% budget	Sin efecto
Menor	2/5	1-2 semanas	5-10%	Menores defects
Moderado	3/5	2-4 semanas	10-25%	Features afectados
Mayor	4/5	1-3 meses	25-50%	Quality significant
Catastrófico	5/5	> 3 meses	> 50%	System failure

Matriz de priorización (Probability × Impact)

Impacto →	1	2	3	4	5
Prob ↓					
5	M	H	H	E	E
4	L	M	H	E	E
3	L	M	M	H	E
2	L	L	M	M	H
1	L	L	L	M	M
L = Low (monitorear) M = Medium (plan de mitigación) H = High (mitigar activamente) E = Extreme (actuar inmediatamente)					

Categorías de riesgos en software

Categoría	Ejemplos	Estrategia típica
Técnico	Deuda técnica, performance, escalabilidad	Spike, PoC, architecture review
Personas	Turnover, knowledge silos, burnout	Pair programming, documentation
Requisitos	Scope creep, requirements ambiguity	Prototyping, early feedback
Schedule	Fechas comprometidas, dependencias	Buffer, parallel tracks
Terceros	Vendor lock-in, API changes	Abstraction, multi-vendor
Seguridad	Vulnerabilidades, compliance	Threat modeling, audits
Organizacional	Budget cuts, reorg, shifting priorities	Stakeholder management

Estrategias de mitigación

Estrategia	Descripción	Ejemplo
Avoid	Eliminar el riesgo cambiando el plan	No usar tecnología experimental
Mitigate	Reducir probabilidad o impacto	Agregar testing, code review
Transfer	Mover el riesgo a tercero	Contratar seguro, usar managed services
Accept	Reconocer y preparar plan de contingencia	Buffer en timeline, runbook
Exploit	Aprovechar la oportunidad (para riesgos positivos)	Adoptar tecnología nueva temprano

Risk Burndown Chart

Mide la reducción de riesgos a lo largo del proyecto.

Template de tracking

Sprint	Riesgos abiertos	Riesgos cerrados	Risk score total
Sprint 1	8	0	32
Sprint 2	7	1	28
Sprint 3	5	3	18
Sprint 4	3	5	10
Sprint 5	1	7	3

Indicadores de salud

Métrica	Target
Risk burndown rate	> 1的风险 por sprint
New risks identified	< 2 por sprint
Overdue mitigations	0
Risk register freshness	Revisado cada sprint

Plantilla de Risk Assessment

```
## Risk: [Nombre del riesgo]

**ID**: R##
**Fecha identificación**: YYYY-MM-DD
**Propietario**: [Nombre]

### Descripción
[Cómo podría ocurrir, contexto]
```

```
### Probabilidad: [1-5]
### Impacto: [1-5]
### Risk Score:  $P \times I = \#$ 

### Mitigación plan
1. [Acción 1] – responsable – fecha
2. [Acción 2] – responsable – fecha

### Contingencia
[Qué hacer si el riesgo materializa]

### Trigger
[Señal de alerta temprana]

### Estado: Active | Monitoring | Mitigated | Closed
```

Anti-patterns

- **Risk blindness**: no documentar riesgos conocidos
- **Analysis paralysis**: evaluar riesgos infinitamente sin actuar
- **Single point of failure**: no tener backup para personas críticas
- **Ignoring positive risks**: no aprovechar oportunidades
- **Risk theater**: documentar riesgos pero no mitigarlos

Stakeholder Management

Gestión de Stakeholders

Identificar, analizar y gestionar a todas las personas o grupos que tienen interés en el resultado del proyecto de software.

Identificación de stakeholders

Categorías principales

Categoría	Ejemplos	Interés típico
Executive	CEO, CTO, VP Engineering	ROI, strategy, timeline
Product	PO, PM, Design Lead	Features, user value, roadmap
Technical	Tech Lead, Architects	Quality, scalability, tech debt
Operations	DevOps, SRE, Support	Stability, monitoring, incidents
Business	Sales, Marketing, Finance	Revenue, competitive advantage
Customer	End users, beta testers	Usability, reliability, features
Compliance	Legal, Security, Audit	Regulatory, data protection
External	Vendors, partners, regulators	Contracts, integration, standards

Power/Interest Grid

Clasifica stakeholders por su poder de influencia y nivel de interés.

Matriz 2x2

		INTERÉS	
		Bajo	Alto
Alto PODER	Alto	KEEP SATISFIED	MANAGE CLOSELY
	Bajo	MONITOR (mínimo)	KEEP INFORMED

Acciones por cuadrante

Cuadrante	Estrategia	Frecuencia	Ejemplo
Manage Closely	Involucrar en decisiones clave	Semanal	CTO, PO
Keep Satisfied	Comunicar progreso, buscar input	Quincenal	VP Sales
Keep Informed	Actualizar regularmente	Mensual	Dev team
Monitor	Check-in mínimo, no gastar tiempo	Trimestral	Otros deptos

Communication Plan

Matriz de comunicación

Audiencia	Formato	Frecuencia	Responsable	Contenido
Executive	Dashboard + Slack	Semanal	EM	KPIs, risks, decisions needed
Product	Sprint review + Backlog	Cada sprint	PO	Features, progress, blockers
Tech team	Standup + Tech review	Diaria/Semanal	TL	Technical decisions, architecture
Customer	Release notes + Demo	Por release	PO/PM	New features, improvements
Operations	Runbook + Monitoring	Continuo	SRE	System health, incidents

Canales de comunicación

Canal	Uso	Audiencia
Slack/Teams	Comunicación rápida, alertas	Todo el equipo
Email	Comunicación formal, decisiones	Stakeholders externos
Jira/Linear	Tracking de trabajo	Equipo técnico
Confluence/Notion	Documentación	Toda la organización
Sprint Review	Demo y feedback	Stakeholders amplios
1:1 Meetings	Feedback individual	Reports directos
All Hands	Comunicación organizacional	Toda la empresa

Stakeholder Mapping Template

Para cada stakeholder clave

Stakeholder: [Nombre / Rol]

Poder: Alto / Medio / Bajo

****Interés**:** Alto / Medio / Bajo
****Actitud**:** Supportive / Neutral / Resistant
****Influencia**:** Alta / Media / Baja

- ### Necesidades
- [Qué necesita saber]
 - [Qué necesita decidir]

- ### Expectativas
- [Qué espera del proyecto]

- ### Estrategia de engagement
- [Cómo involucrarlo]
 - [Frecuencia de comunicación]
 - [Canal preferido]

- ### Riesgo
- [Qué pasa si lo ignoramos]

Gestión de resistencia

Tipos de resistencia y respuestas

Tipo	Señales	Respuesta
Fear of change	"Siempre lo hemos hecho así"	Demostrar beneficios, quick wins
Lack of trust	Micro-managing, skepticism	Transparencia, datos, consistencia
Competing priorities	"No tengo tiempo"	Mostrar alineación con sus OKRs
Technical disagreement	"Esa tecnología no funciona"	Spike, PoC, data-driven
Political	"Mi equipo no puede"	Escalation, executive sponsorship

Reporting para stakeholders

Dashboard ejecutivo (ejemplo semanal)

Métrica	Valor	Trend	Target
Sprint velocity	42 SP	↑	40 SP
Features completadas	5/8	→	6/8
Bug open	12	↓	< 15
Tech debt items	8	→	< 10
On-time delivery	92%	↑	> 90%

Métrica	Valor	Trend	Target
Risk score	8 (Low)	↓	< 15

Formato de status report

1. Resumen ejecutivo (1–2 oraciones)
2. Logros de la semana
3. Próximos pasos
4. Riesgos y blockers
5. Decisiones necesarias
6. Métricas clave

Anti-patterns

- **Stakeholder vacuum:** no comunicar hasta que preguntan
- **Info dump:** enviar toda la información sin filtrar
- **Ignoring resistencia:** asumir que todos están alineados
- **One-size-fits-all:** mismo reporte para audiencias diferentes
- **Only upward reporting:** solo comunicar hacia arriba, no hacia abajo

Budget Cost

Presupuesto y Costos en Software

Entender los costos de construir, operar y mantener software es esencial para la toma de decisiones de negocio.

Componentes del costo de software

Desglose de costos

Categoría	% típico del TCO	Ejemplos
People	60-70%	Salarios, benefits, training
Infrastructure	10-20%	Cloud, hosting, licenses
Tools	5-10%	IDEs, CI/CD, monitoring
Process	5-10%	Compliance, audits, training
Maintenance	15-25% anual	Bug fixes, updates, support

Costo por fase del proyecto

Fase	% del budget total	Duración típica
Discovery & Planning	5-10%	2-4 semanas
Design & Architecture	10-15%	2-6 semanas
Development	40-50%	3-12 meses
Testing & QA	15-20%	Paralelo a dev
Deployment & Launch	5-10%	1-4 semanas
Post-launch support	5-10% (anual)	Continuo

Total Cost of Ownership (TCO)

Incluye todos los costos durante la vida útil del software.

Fórmula TCO

TCO = Development + Infrastructure + Operations + Maintenance + Opportunity Cost

TCO anual = (Salarios × FTE) + (Cloud × 12) + (Licencias × 12) + (Support cost)

Ejemplo de cálculo TCO (3 años)

Componente	Año 1	Año 2	Año 3	Total
Development (5 devs)	\$400K	\$200K	\$150K	\$750K
Cloud infrastructure	\$60K	\$72K	\$85K	\$217K
Tools & licenses	\$25K	\$25K	\$25K	\$75K
Operations (0.5 FTE)	\$50K	\$50K	\$50K	\$150K
Maintenance & support	\$40K	\$60K	\$80K	\$180K
Total	\$575K	\$407K	\$390K	\$1.372M

Cost of Delay (CoD)

El costo de NO entregar un feature. Es la métrica más importante para priorización.

Fórmula

CoD = (Value per time unit) × (Duration of delay)

Componentes del CoD

Componente	Descripción	Ejemplo
Revenue impact	Ingresos perdidos por no tener la feature	\$50K/mes en ventas perdidas
User impact	Usuarios que no adoptan o churn	200 usuarios/mes perdidos
Strategic cost	Ventaja competitiva perdida	Competidor lanza primero
Technical cost	Deuda técnica que crece	Costo de fix crece 30%/mes
Risk cost	Riesgo que aumenta sin acción	Compliance deadline approaching

Ejemplo práctico

Feature: Sistema de pagos con Stripe

Value per month: \$120K (revenue directa)

Delay: 2 meses

CoD = \$120K × 2 = \$240K

Si el costo de construir = \$80K

ROI = (\$240K - \$80K) / \$80K = 200%

Estimación de costos de proyecto

By team composition

Rol	Costo mensual (US)	% del equipo
Tech Lead	\$15K-20K	1
Senior Dev	\$12K-18K	2-3
Mid-level Dev	\$8K-12K	3-4
Junior Dev	\$5K-8K	1-2
QA Engineer	\$8K-12K	1-2
DevOps	\$10K-15K	0.5-1
Designer	\$8K-12K	0.5-1
Scrum Master / EM	\$12K-16K	0.5

Estimación por tipo de proyecto

Tipo	Equipo	Duración	Costo total
MVP / Proof of Concept	3-4 devs	2-3 meses	\$80K-150K
MVP completo	5-7 devs	4-6 meses	\$300K-600K
Producto mediano	8-12 devs	6-12 meses	\$800K-2M
Plataforma enterprise	15-30 devs	12-24 meses	\$3M-10M
Migración legacy	5-10 devs	6-18 meses	\$500K-2M

Budget Tracking

Métricas financieras

Métrica	Fórmula	Target
Cost Performance Index (CPI)	EV / AC	> 1.0
Schedule Performance Index (SPI)	EV / PV	> 1.0
Budget variance	(actual - budget) / budget	< 10%
Burn rate	monthly spend / total budget	Según plan
ROI	(gain - cost) / cost	> 100%

Earned Value Management

PV (Planned Value): valor del trabajo planificado hasta hoy
EV (Earned Value): valor del trabajo completado hasta hoy
AC (Actual Cost): costo real incurrido hasta hoy

$CPI = EV / AC \rightarrow > 1 = \text{under budget}, < 1 = \text{over budget}$
 $SPI = EV / PV \rightarrow > 1 = \text{ahead of schedule}, < 1 = \text{behind}$

Cost Optimization Strategies

Estrategia	Ahorro típico	Trade-off
Cloud right-sizing	20-40% infra	Requiere monitoring
Open source	30-50% tools	Soporte propio
Outsourcing nearshore	30-50% people	Communication overhead
Automation	20-30% operations	Initial investment
Technical debt paydown	15-25% maintenance	Upfront cost

Anti-patterns

- **Budget theater:** tracking sin action sobre variance
- **Ignoring maintenance:** no incluir costos post-launch
- **Sunk cost fallacy:** continuar por haber invertido mucho
- **Hidden costs:** no contar training, onboarding, context switching
- **Optimistic bias:** asumir mejor caso siempre

Contratacion Modelos

Modelos de Contratación

Elegir el modelo correcto de contratación impacta directamente en costo, riesgo, flexibilidad y velocidad de entrega.

Comparación general

Modelo	Riesgo (cliente)	Flexibilidad	Predictibilidad	Overhead admin
Fixed Price	Alto	Baja	Alta	Baja
Time & Materials	Medio	Alta	Baja	Media
Dedicated Team	Medio	Muy alta	Media	Media
Staff Augmentation	Medio-bajo	Media	Baja	Baja
Managed Service	Bajo	Baja	Alta	Baja
Outsourcing	Variable	Variable	Media	Alta

Fixed Price

El vendor se compromete a entregar un alcance definido por un precio fijo.

Cuándo usar

Situación	Recomendación
Alcance 100% definido	
Proyecto corto (< 3 meses)	
Requisitos estables y documentados	
Alcance不确定	×
Proyecto ágil iterativo	×
Tecnología nueva/experimental	×

Estructura típica

- Fixed Price Contract
- Scope document (SOW)
 - Milestones & deliverables
 - Payment schedule (por milestone)
 - Change request process

- └ Penalty clauses (delayed delivery)
- └ Acceptance criteria

Pros y contras

Pros	Contras
Predictibilidad de budget	Cambios son caros (change requests)
Vendor asume riesgo de overruns	Vendor puede recortar calidad
Simple de administrar	Incentivo a entregar mínimo viable
Claro para stakeholders	Requiere documentación extensa

Time & Materials (T&M)

El cliente paga por tiempo (horas/días) y materiales consumidos.

Cuándo usar

Situación	Recomendación
Alcance evoluciona (ágil)	
Proyecto largo (> 3 meses)	
Necesidad de flexibilidad	
Budget fijo estricto	×
Sin capacidad de supervisión	×

Estructura típica

- T&M Contract
- └ Rate card (por rol/nivel)
 - └ Monthly invoices (horas reales)
 - └ Weekly timesheet reporting
 - └ Right to audit
 - └ Cap mensual (optional)
 - └ Notice period (30-60 días)

Rates típicos (referencia US/Europa)

Rol	Nearshore	Offshore	Onsite
Senior Dev	\$60-100/hr	\$35-60/hr	\$120-180/hr
Mid Dev	\$40-70/hr	\$20-40/hr	\$80-120/hr

Rol	Nearshore	Offshore	Onsite
QA	\$35-60/hr	\$15-30/hr	\$70-100/hr
DevOps	\$50-90/hr	\$30-50/hr	\$100-150/hr

Dedicated Team

Un equipo completo asignado exclusivamente al cliente, bajo gestión del vendor.

Cuándo usar

Situación	Recomendación
Proyecto largo (> 6 meses)	
Necesidad de equipo estable	
Producto en crecimiento continuo	
Proyecto puntual corto	✗
Sin capacidad de management	✗

Estructura típica

- Dedicated Team
 - Team composition acordada
 - Monthly fee (fijo por seat)
 - Client manages priorities & backlog
 - Vendor manages hiring & retention
 - Ramp-up period (1-2 meses)
 - Minimum commitment (6-12 meses)

Staff Augmentation

Contratar individuos del vendor para integrarlos al equipo existente del cliente.

Cuándo usar

Situación	Recomendación
Fill skill gaps específicos	
Escalar equipo temporalmente	
Cobertura vacaciones/permuta	
Equipo completo nuevo	✗ (mejor Dedicated Team)
Sin gestión interna fuerte	✗

Comparación con otros modelos

Aspecto	Staff Aug	Dedicated Team	Outsourcing
Control del cliente	Alto	Alto	Bajo
Management	Cliente	Compartido	Vendor
Onboarding	Cliente	Vendor	Vendor
Retention risk	Cliente	Vendor	Vendor
Costo	Medio	Medio-alto	Bajo-medio

Outsourcing completo

El vendor asume todo: equipo, proceso, entrega.

Cuándo usar

Situación	Recomendación
Producto no core	
Startup sin equipo técnico	
Proyecto con alcance claro	
Producto core estratégico	✗
Necesidad de innovación rápida	✗

Framework de decisión

```
¿El alcance está 100% definido?  
├─ Sí → ¿Budget fijo?  
│   ├── Sí → Fixed Price  
│   └─ No → T&M  
└─ No → ¿Necesitas equipo dedicado?  
    ├── Sí → ¿Más de 6 meses?  
    │   ├── Sí → Dedicated Team  
    │   └─ No → Staff Augmentation  
    └─ No → T&M con cap mensual
```

Risk Allocation Matrix

Riesgo	Fixed Price	T&M	Dedicated Team	Outsourcing
Scope creep	Vendor	Cliente	Cliente	Compartido
Delay	Vendor	Cliente	Cliente	Vendor

Riesgo	Fixed Price	T&M	Dedicated Team	Outsourcing
Quality issues	Vendor	Compartido	Compartido	Vendor
Cost overrun	Vendor	Cliente	Cliente	Vendor
Key person dependency	Compartido	Cliente	Vendor	Vendor
Technology risk	Compartido	Cliente	Cliente	Vendor

Best Practices

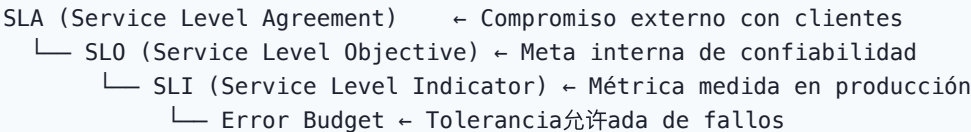
- Incluir **cláusula de salida** en todos los contratos
- Definir **SLAs claros** para calidad y timeliness
- Usar **proof of concept** antes de comprometer modelo completo
- Establecer **checkpoints** mensuales independientemente del modelo
- Documentar **acceptance criteria** antes de cada pago
- Revisar el modelo cada 6 meses: ¿sigue siendo el adecuado?

Sla Slo Error Budgets

SLAs, SLOs y Error Budgets

Definir y gestionar niveles de servicio es fundamental para balancear confiabilidad con velocidad de innovación.

Jerarquía de niveles de servicio



SLI (Service Level Indicator)

La métrica concreta que mide el nivel de servicio.

SLIs comunes

SLI	Fórmula	Ejemplo de medición
Disponibilidad	$\text{exit requests} / \text{total requests}$	99.95% uptime
Latencia	$\text{requests} < \text{threshold} / \text{total requests}$	99% < 200ms
Throughput	$\text{successful requests} / \text{time}$	10K req/min
Error rate	$\text{errors} / \text{total requests}$	< 0.1%
Durabilidad	$\text{data retained} / \text{data written}$	99.999999%

Ejemplo de SLI con datos

```
# Disponibilidad en 30 días
total_requests = 10_000_000
failed_requests = 2_500
availability = (total_requests - failed_requests) / total_requests
# = 99.975%

# Latencia p99
latencies = [...] # 10M samples
p99 = percentile(latencies, 99)
# = 180ms (target: < 200ms ✓)
```

SLO (Service Level Objective)

El target interno que el equipo se compromete a mantener.

SLOs típicos por tipo de servicio

Tipo de servicio	Disponibilidad	Latencia p99	Error rate
Mission-critical (pagos)	99.99%	< 100ms	< 0.01%
Business-critical (API pública)	99.95%	< 200ms	< 0.1%
Standard (internal tools)	99.9%	< 500ms	< 0.5%
Best effort (analytics)	99.5%	< 1s	< 1%

Plantilla de SLO

```
## SLO: [Nombre del servicio]

**Período de medición**: Rolling 30 days
**SLO target**: 99.95% disponibilidad

### SLI elegido
[Descripción de cómo se mide]

### Justificación del target
- Requisito del cliente: 99.9%
- Competitive benchmark: 99.95%
- Costo de mayor confiabilidad: $X/mes

### Consecuencias de violar
- Customer impact assessment
- Revenue impact
- Brand impact
```

Error Budget

La cantidad máxima de fallos permitidos antes de que el SLO se incumpla.

Cálculo del Error Budget

```
Error Budget = 1 - SLO

SLO 99.9% → Error Budget = 0.1% = 43.8 minutos/mes
SLO 99.95% → Error Budget = 0.05% = 21.9 minutos/mes
SLO 99.99% → Error Budget = 0.01% = 4.38 minutos/mes
```

Budget por dimensiones

SLO	Time Budget	Request Budget (10M req/mes)
99.9%	43.8 min/mes	10,000 failed requests
99.95%	21.9 min/mes	5,000 failed requests
99.99%	4.38 min/mes	1,000 failed requests

Error Budget Policy

```
## Error Budget Policy

### Consumo < 50% (Saludable)
- Deploy libremente
- Innovation sprints permitidos
- Tech debt paydown permitido

### Consumo 50-80% (Precaución)
- Deploy con approvals adicionales
- Focus en testing y observabilidad
- Reducir scope de features nuevas

### Consumo 80-100% (Crítico)
- Solo deploy de bug fixes y hotfixes
- Freeze de features nuevas
- Todas las manos en deck: reliability sprint

### Consumo 100% (Violación)
- Incidente mayor declarado
- Post-mortem obligatorio
- Plan de remediación en 48 horas
- Review con management
```

SLA (Service Level Agreement)

Compromiso contractual externo con consecuencias medibles.

Elementos de un SLA

Elemento	Descripción	Ejemplo
Scope	Servicios cubiertos	API pública, dashboard
Metrics	SLIs medidos	Disponibilidad, latencia
Targets	SLOs contractuales	99.9% uptime
Reporting	Frecuencia y formato	Monthly report
Remedies	Penalizaciones por incumplimiento	Credit 10% del fee

Elemento	Descripción	Ejemplo
Exclusions	Qué no cubre	Maintenance windows

Ejemplo de SLA structure

```
## Service Level Agreement – API Service

### 1. Disponibilidad
Provider garantiza 99.95% de disponibilidad mensual.

### 2. Medición
Basado en health check endpoint, medido cada 60 segundos.

### 3. Maintenance Windows
Primer martes de cada mes, 02:00–04:00 UTC.
No contabiliza para availability.

### 4. Reporting
Dashboard público actualizado en tiempo real.
Reporte mensual enviado por email.

### 5. Remedies
| Availability | Credit |
|-----|-----|
| 99.9% – 99.95% | 5% monthly fee |
| 99.0% – 99.9% | 10% monthly fee |
| < 99.0% | 25% monthly fee |
```

Implementación práctica

Stack de medición

Componente	Herramienta	Propósito
Metrics	Prometheus, Datadog	Collect SLIs
Alerting	PagerDuty, OpsGenie	Alert on SLO burn
Dashboard	Grafana, Datadog	Visualize error budget
Reporting	Custom scripts	Monthly SLA reports
Incident tracking	Jira, PagerDuty	Track incidents

Burn rate alerting

$$\text{Burn Rate} = \text{actual_error_rate} / \text{SLO_error_rate}$$

Burn Rate 14.4x → Alert en 1 hora (budget agota en 1 día)
Burn Rate 6x → Alert en 6 horas
Burn Rate 3x → Alert en 1 día
Burn Rate 1x → Alert en 3 días (budget agota en 30 días)

Anti-patterns

- **100% SLO**: objetivo irreal, crea miedo a deploy
- **Vanity SLOs**: metrics que nadie monitorea
- **No consequences**: error budget que no afecta decisiones
- **Ignoring maintenance windows**: no descontar maintenance planificado
- **Gaming metrics**: optimizar el SLI sin mejorar el servicio real

Okr Software

OKRs para Equipos de Software

OKRs (Objectives and Key Results) alinean el trabajo técnico con los objetivos de negocio, midiendo outcomes en lugar de outputs.

Estructura de un OKR

```
Objective: [Cualitativo, inspirador, alcanzable en 1 quarter]
├─ KR1: [Cuantificable, medible, ambicioso]
├─ KR2: [Cuantificable, medible, ambicioso]
└─ KR3: [Cuantificable, medible, ambicioso]
```

Reglas de calidad

Componente	Debe ser	No debe ser
Objective	Inspirador, ambicioso, cualitativo	Numérico, vago, incremental
Key Result	Cuantificable, medible, con timeline	Actividad, binario, no medible
Key Result	Outcome (resultado)	Output (actividad)

Buenos vs malos ejemplos

❌ Malo	Bueno	Tipo
"Mejorar el testing"	"Aumentar code coverage del 60% al 85%"	KR
"Deploy más rápido"	"Reducir deployment frequency de semanal a diaria"	KR
"Hacer code review"	"100% de PRs revisados en < 4 horas"	KR
"Ser más confiables"	"Reducir incidentes P1 de 5/mes a 1/mes"	KR

OKRs para Engineering — Ejemplos

Q1: Developer Experience

```
O: Hacer que los desarrolladores sean 50% más productivos
├─ KR1: Reducir CI build time de 15min a 7min
├─ KR2: Reducir time-to-first-deploy de 2 días a 4 horas
└─ KR3: Developer satisfaction score de 6.5 a 8.5/10
```

Q1: Reliability

- 0: Alcanzar confiabilidad de nivel enterprise
 - KR1: Disponibilidad de 99.9% a 99.95%
 - KR2: MTTR de 45min a 15min
 - KR3: 0 incidents P1 por quarter (vs 3 el quarter anterior)

Q1: Technical Excellence

- 0: Eliminar la deuda técnica crítica del sistema
 - KR1: Reducir critical vulnerabilities de 24 a 0
 - KR2: Migrar 100% de servicios a Kubernetes
 - KR3: Code coverage > 80% en todos los servicios core

Q1: Delivery Speed

- 0: Entregar valor al usuario 3x más rápido
 - KR1: Reducir cycle time promedio de 8 días a 3 días
 - KR2: Aumentar deployment frequency de semanal a diaria
 - KR3: Lead time for changes < 1 día (DORA metric)

Cadencia de OKRs

Actividad	Frecuencia	Participantes
Setting OKRs	Quarterly (antes del quarter)	EM + Tech Lead + PO
Check-in	Semanal	Equipo
Mid-quarter review	Mitad del quarter	EM + Stakeholders
Scoring	End of quarter	EM + Equipo
Retrospective	Post-scoring	Equipo completo

Scoring scale

Score	Significado
1.0	Logrado completamente
0.7	Progreso significativo (target era ambicioso)
0.5	Progreso moderado
0.3	Poco progreso

Score	Significado
0.0	Sin progreso

Target: promedio de 0.6-0.7 (ambicioso pero achievable)

Common Mistakes

Error	Descripción	Cómo evitarlo
Too many OKRs	> 5 objectives por quarter	Máximo 3-5 objectives
Output as KR	"Implementar feature X" como KR	Medir outcome, no actividad
Not ambitious enough	Todos score 1.0	Target: 0.7 average
Set and forget	No check-ins durante el quarter	Weekly check-ins
Cascading OKRs	OKRs individuales = team OKRs	Team OKRs, no individuales
Ignoring tech debt	Solo business outcomes	Incluir 1 OKR técnico

OKR Alignment

Ejemplo de alignment vertical

Company OKR:
O: Ser líder en mercado X
KR: \$10M ARR

└─ Product OKR:
O: Mejorar activation rate
KR: 30% → 50% activation

└─ Engineering OKR:
O: Reducir time-to-value para usuarios
KR: Onboarding flow < 5 minutos

└─ Marketing OKR:
O: Aumentar awareness
KR: 100K monthly visitors

OKR vs KPI

Aspecto	OKR	KPI
Propósito	Alcanzar nuevo nivel	Mantener nivel actual
Ambición	70% achievement = éxito	100% achievement = éxito

Aspecto	OKR	KPI
Cambio	Cambia cada quarter	Estable durante largo tiempo
Ejemplo	"Reducir latency 50%"	"Mantener 99.9% uptime"
Mindset	Growth / Transformación	Operations / Mantenimiento

Anti-patterns

- **OKRs como performance review:** genera gaming de métricas
- **Micromanagement via OKRs:** demasiado granular
- **No psychological safety:** miedo a no alcanzar targets
- **OKRs estáticos:** no ajustar durante el quarter
- **Bottom-up only:** OKRs sin alineación con strategy

Portfolio Management

Portfolio Management

Priorizar iniciativas de software a nivel de portfolio usando frameworks basados en datos, no en intuición.

Frameworks de priorización

Comparación rápida

Framework	Mejor para	Complejidad	Sesgo principal
RICE	Features de producto	Baja	Overconfidence en estimates
WSJF	Agile at scale	Media	Difficulty estimating delay
MoSCoW	Scope negotiation	Baja	Everything becomes "Must"
Value vs Effort	Quick prioritization	Baja	Oversimplification
Kano	Feature categorization	Media	Requires user research
Cost of Delay	Business case	Alta	Hard to quantify
OpScore	Resource allocation	Alta	Complex to maintain

RICE Score

Developed by Intercom. Prioriza por impacto relativo ajustado por esfuerzo.

Fórmula

$$\text{RICE} = (\text{Reach} \times \text{Impact} \times \text{Confidence}) / \text{Effort}$$

Factor	Escala	Descripción
Reach	Users/quarter	Cuántos usuarios afecta
Impact	0.25, 0.5, 1, 2, 3	Impacto por usuario
Confidence	100%, 80%, 50%, 20%	Certeza de los estimates
Effort	Person-months	Trabajo del equipo

Ejemplo RICE

Feature	Reach	Impact	Confidence	Effort	RICE
Search v2	5000	3	80%	2	6000
SSO integration	2000	2	50%	3	667
Export to PDF	8000	0.5	100%	1	4000
Dark mode	10000	0.25	100%	0.5	5000

Ranking: Search v2 (6000) > Dark mode (5000) > Export (4000) > SSO (667)

WSJF (Weighted Shortest Job First)

Framework de SAFe. Prioriza por costo de retraso dividido por duración.

Fórmula

WSJF = Cost of Delay / Job Size

Componentes del Cost of Delay

CoD = User/Business Value + Time Criticality + Risk Reduction

User/Business Value: 1-10 (1=baja, 10=crítica)
Time Criticality: 1-10 (1= sin fecha, 10=deadline mañana)
Risk Reduction: 1-10 (1=no reduce riesgo, 10=previene catástrofe)

Job Size (Fibonacci)

Tamaño	Duración approx
1	1 día
2	2-3 días
3	1 semana
5	2 semanas
8	1 mes
13	2+ meses

Ejemplo WSJF

Iniciativa	UV	TC	RR	CoD	Size	WSJF
Fix checkout bug	10	10	5	25	2	12.5

Iniciativa	UV	TC	RR	CoD	Size	WSJF
New dashboard	7	3	2	12	5	2.4
API v2	8	5	7	20	8	2.5
Dark mode	3	1	1	5	3	1.7

Ranking: Fix checkout bug (12.5) > API v2 (2.5) > Dashboard (2.4) > Dark mode (1.7)

MoSCoW

Categoriza features en 4 niveles de prioridad.

Categoría	Significado	% típico del scope
Must have	Sin esto, el sistema no funciona	60%
Should have	Importante pero no crítico	20%
Could have	Nice to have, si hay tiempo	15%
Won't have (this time)	Fuera de scope este ciclo	5%

Regla de MoSCoW

- Máximo **60%** en Must Have
- Si todo es Must, nada es Must
- Won't have = decisión explícita, no olvido

Value vs Effort Matrix

	ESFUERZO	
	Bajo	Alto
ALTO VALOR	QUICK WINS (hacer)	MAJOR PROJECTS (planear)
BAJO VALOR	FILL-INS (si hay tiempo)	THANKLESS TASKS (evitar)

Portfolio Balancing

Framework de categorías

Categoría	% del portfolio	Horizonte	Riesgo
Run (mantener)	40-60%	Corto	Bajo
Grow (expandir)	20-30%	Medio	Medio
Transform (innovar)	10-20%	Largo	Alto

Ejemplo de distribución

Portfolio Total: 100 points (capacity del equipo)

RUN (50 pts):

- └ Bug fixes & maintenance: 20 pts
- └ Tech debt reduction: 15 pts
- └ Operational improvements: 15 pts

GROW (30 pts):

- └ Feature enhancements: 20 pts
- └ Platform improvements: 10 pts

TRANSFORM (20 pts):

- └ New product exploration: 10 pts
- └ Architecture modernization: 10 pts

Portfolio Review Meeting

Frecuencia	Participantes	Agenda
Mensual	EM, PO, Tech Lead, PM	Review status, rebalance
Quarterly	+ Leadership, Finance	Strategic alignment, budget

Agenda típica (90 min)

1. Portfolio health dashboard (10 min)
2. Active initiatives status (20 min)
3. Completed initiatives outcomes (15 min)
4. New initiatives candidates (20 min)
5. Reprioritization discussion (15 min)
6. Resource allocation decisions (10 min)

Anti-patterns

- **HiPPO**: Highest Paid Person's Opinion drives decisions
- **Sunk cost**: continuing because of past investment
- **Feature factories**: measuring output not outcome

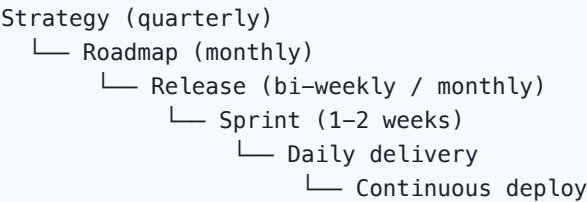
- **No kill criteria:** launching but never stopping initiatives
- **Checkbox mentality:** completing without measuring value
- **Analysis paralysis:** over-analyzing without deciding

Delivery Ritmos

Ritmos de Delivery

Definir cadencias claras de entrega permite predecibilidad, reduce anxiety y crea hábitos en el equipo.

Ciclo de delivery



Sprint Cadence

Duraciones típicas

Duración	Pros	Contras	Cuándo usar
1 semana	Feedback rápido, urgencia baja	Overhead de ceremony	Equipos maduros, Kanban
2 semanas	Balance ideal	Estándar de industria	La mayoría de equipos
3 semanas	Más tiempo para features complejas	Feedback más lento	Equipos con features grandes
4 semanas	Mucho tiempo de desarrollo	Mucha puede cambiar en 4 semanas	Raro, evitar si es posible

Eventos del sprint (2 semanas)

Evento	Duración	Frecuencia	Propósito
Sprint Planning	2h	Inicio del sprint	Definir scope del sprint
Daily Standup	15 min	Diario	Sincronización
Backlog Refinement	1h	Mitad del sprint	Preparar próximo sprint
Sprint Review	1h	Final del sprint	Demo y feedback
Sprint Retrospective	1h	Final del sprint	Mejora continua

Capacidad del sprint

$\text{Sprint capacity} = (\text{días del sprint} - \text{holidays} - \text{PTO} - \text{meetings}) \times \text{team size}$

Ejemplo (2 semanas, 5 devs):

Días hábiles: 10

Holidays: 1

PTO: 0.5 (promedio)

Meetings: 1 (sprint ceremonies)

$\text{Effective days} = 10 - 1 - 0.5 - 1 = 7.5 \text{ días/dev}$

$\text{Capacity} = 7.5 \times 5 = 37.5 \text{ días de desarrollo}$

Release Cadence

Modelos de release

Modelo	Frecuencia	Ejemplo	Best for
Continuous Delivery	Cada commit	Netflix, Google	Products web, SaaS
Weekly release	Cada semana	Shopify, GitHub	Active products
Bi-weekly	Cada 2 sprints	Many startups	Growing products
Monthly	Cada month	Enterprise apps	Stable products
Quarterly	Cada 3 meses	Large enterprises	Regulated industries
Big Bang	Cuando está listo	V1 launches	New products

Release strategy

Estrategia	Descripción	Rollback
Trunk-based	Deploy from main	Feature flags
Release branches	Branch per release	Revert commit
GitFlow	Develop + release branches	Merge back
Environment promotion	Dev → Staging → Prod	Promote previous
Blue-Green	Two identical environments	Switch traffic
Canary	Gradual rollout	Route back

Increment vs Release

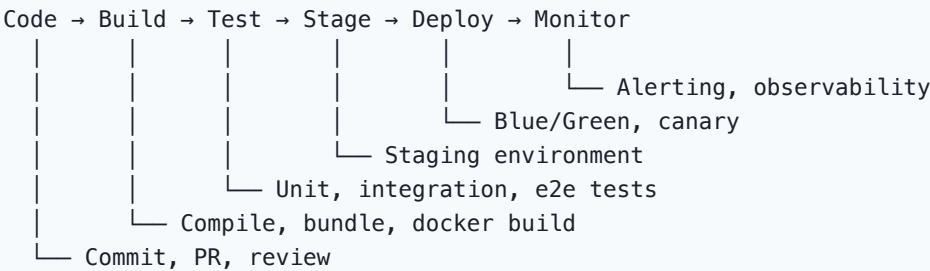
Concepto	Definición	Frecuencia
Increment	Nuevo valor añadido al producto	Cada sprint

Concepto	Definición	Frecuencia
Release	Increment puesto a disposición del usuario	Variable
Deployment	Poner código en producción	Variable

Sprint 1: Increment A (no released – feature flag off)
 Sprint 2: Increment B (A released – flag on, B flag off)
 Sprint 3: Increment C (A+B+C released together)

Continuous Delivery

Pipeline stages



Métricas DORA

Métrica	Low performer	Medium	High performer
Deployment frequency	Monthly	Weekly	On-demand (multiple/day)
Lead time for changes	1-6 months	1 week-1 month	< 1 day
Change failure rate	46-60%	16-30%	0-15%
MTTR	1 week-1 month	1 day-1 week	< 1 hour

Delivery Anti-patterns

Anti-pattern	Problema	Solución
Release train	Todo sale junto, beaucoup de bugs	Feature flags, smaller releases
Deploy freeze	No se deploja por miedo	Improbear testing, canary
Sprint hero	1 persona hace todo al final	Pair programming, WIP limits
Carry-over	Stories que nunca terminan	Smaller stories, capacity planning
Definition of Done vago	"Casi listo" = no listo	DoD estricto y verificado

Anti-pattern	Problema	Solución
Demo-driven development	Features para demo, no para usuario	User feedback loops

Cadencia recomendada por tipo de producto

Tipo de producto	Sprint	Release	Deploy
SaaS B2B	2 semanas	Monthly	Continuous
SaaS B2C	2 semanas	Weekly	Continuous
Mobile app	2 semanas	Bi-weekly	App store cycle
Enterprise	2-3 semanas	Quarterly	Scheduled
API platform	2 semanas	As needed	Continuous
Embedded/IoT	3-4 semanas	Quarterly	OTA updates

Rituals checklist

- ☐ Sprint Planning: scope definido y capacity verificado
- ☐ Daily Standup: blockers identificados y assigned
- ☐ Refinement: top 10 items ready para próximo sprint
- ☐ Sprint Review: demo real con datos de uso
- ☐ Sprint Retro: 3 acciones concretas para mejora
- ☐ Release notes: escritas y comunicadas
- ☐ Monitoring: dashboard actualizado post-deploy

README

Gestión de Proyectos

Planificación, estimación, riesgos, stakeholders y delivery de software. Herramientas y marcos para entregar valor de forma predecible y escalable.

Contenido

#	Archivo	Tema	Nivel	
01	[[../10-Gestion-Proyectos/01-planificacion-agil	Planificación Ágil]]	Roadmapping, PI Planning, release planning	Intermedio
02	[[../10-Gestion-Proyectos/02-estimacion-tecnicas	Técnicas de Estimación]]	Planning poker, T-shirt sizing, reference class	Intermedio
03	[[../10-Gestion-Proyectos/03-story-points-tshirt	Story Points vs T-Shirt Sizing]]	Pros, cons, cuándo usar cada uno	Principiante
04	[[../10-Gestion-Proyectos/04-backlog-grooming	Backlog Grooming / Refinement]]	INVEST, story mapping, slicing	Intermedio
05	[[../10-Gestion-Proyectos/05-risk-management	Gestión de Riesgos]]	Risk register, probabilidad/impacto, mitigación	Intermedio
06	[[../10-Gestion-Proyectos/06-stakeholder-management	Gestión de Stakeholders]]	Power/interest grid, communication plan	Intermedio
07	[[../10-Gestion-Proyectos/07-budget-cost	Presupuesto y Costos]]	Cost estimation, TCO, cost of delay	Avanzado
08	[[../10-Gestion-Proyectos/08-contratacion-modelos	Modelos de Contratación]]	T&M, Fixed Price, Dedicated Team, Outsourcing	Intermedio
09	[[../10-Gestion-Proyectos/09-sla-slo-error-budgets	SLAs, SLOs y Error Budgets]]	Definición, cálculo, políticas	Avanzado
10	[[../10-Gestion-Proyectos/10-okr-software	OKRs para Equipos de Software]]	Setting, examples, common mistakes	Intermedio
11	[[../10-Gestion-Proyectos/11-portfolio-management	Portfolio Management]]	RICE, WSJF, Cost of Delay, priorización	Avanzado
12	[[../10-Gestion-Proyectos/12-delivery-ritmos	Ritmos de Delivery]]	Sprints, increments, release cadence, CD	Intermedio

Secciones relacionadas

- 01 — Fundamentos

- 02 — Estructura
- 03 — Procesos
- 06 — Métricas
- 11 — Cultura y Equipo